

第13課

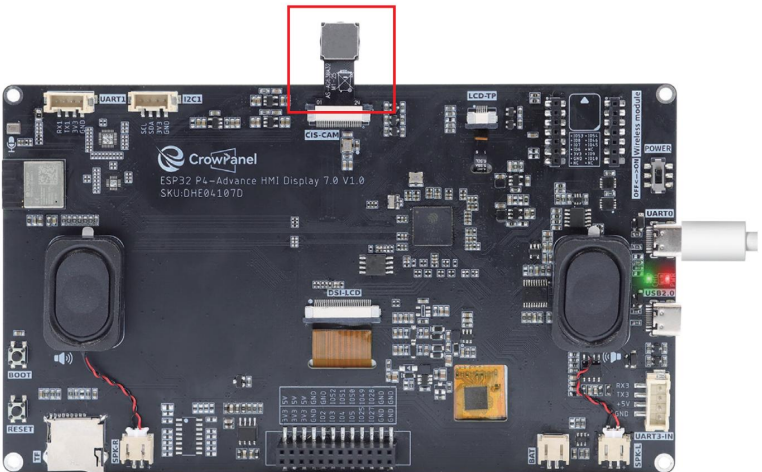
カメラリアルタイム

導入

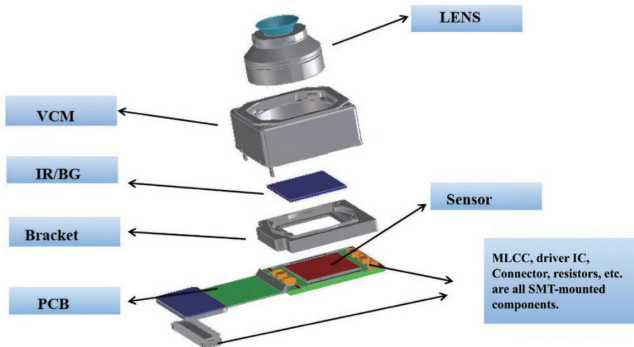
このレッスンでは、カメラを起動して、Advance-P4の画面にカメラ映像をリアルタイムで表示する方法を説明します。

このレッスンで使用するハードウェア

Advance-P4のカメラ



カメラの概略図



まず、レンズは光の「収集器」としての役割を果たします。その光学構造は外部からの光を捉え、曲率やその他の設計上の特徴によって光を収束させ、後続の画像処理のための基本的な光信号を生成します。

次に、ボイスコイルモーター（VCM）はオートフォーカスにおいて重要な役割を果たします。回路からの制御信号に基づいて、電磁誘導の原理を利用してレンズを一定範囲内で正確に駆動します。レンズとイメージセンサー（センサー）間の距離を変化させることで、光の焦点を調整し、撮影対象物がセンサー上に鮮明に結像されるようにします。光がセンサーに到達する前に、IRカット/ブルーガラスフィルタ（IR/BG）が光をフィルタリングします。IRカットフィルタは赤外線を遮断します。赤外線は可視光による結像に干渉し、色の歪みを引き起こす可能性があるためです。ブルーガラスフィルタは赤外線を遮断するだけでなく、迷光の侵入も低減し、光の純度をさらに向上させ、後段のセンサーが受け取る光が、より正確な色と鮮明さを備えた画像の形成に適したものとなるようにします。

次に、中核部品であるイメージセンサー（センサー）の表面には、フォトダイオードなどの感光素子が配置されており、受信した光信号を電気信号に変換する。光の強度が異なると、感光素子はシーンの明るさや色などの情報に対応する、大きさの異なる電気信号を生成する。

最後に、プリント基板（PCB）上に実装された積層セラミックコンデンサ（MLCC）、ドライバ集積回路（ドライバIC）、コネクタ、抵抗器などの部品が、回路接続を介して完全な信号処理および伝送システムを構成する。

ドライバICは、センサーによって生成された電気信号の増幅やアナログ-デジタル変換などの前処理を担当し、アナログ電気信号をデジタル信号に変換します。MLCCなどのコンデンサや抵抗器は、電圧を安定させ、ノイズを除去し、回路の安定動作を保証します。

このように処理されたデジタル信号は、コネクタを介して後続のデバイス（携帯電話やカメラのメイン制御チップなど）に送信され、最終的に復号化されて、私たちが目にするデジタル画像として表示されます。

動作効果図

コードを実行すると、カメラからのリアルタイム映像がAdvance-P4の画面に表示されます。



重要な説明

- さて、このレッスンの主な焦点は、カメラの使い方と、カメラの映像を画面に表示する方法です。
- ここでは、新しいコンポーネント「bsp_camera」を用意します。
- このコンポーネントの主な機能は以下のとおりです。
 - カメラのSCCB (I2C)通信バスとESP32のCSIハードウェアインターフェースの初期化を担当し、カメラの低レベルドライバの起動と設定を完了する。
 - カメラは標準のV4L2プロトコルに基づいて起動および設定され、LVGLと互換性のあるRGB565フォーマットに自動的に設定され、画像反転などの機能をサポートします。
 - マルチフレーム画像キャッシュを管理し、ダブルバッファリングメカニズムをサポートします。整列されたメモリ領域を自動的に割り当て、キャッシュのエンキュー、デキュー、および循環使用を完了します。
 - フレームデータコールバックインターフェースを提供します。画像フレームが収集されるたびに、上位レイヤーに自動的に通知され、ビデオ取得とビジネスロジックの分離が実現されます。

- 独立したビデオ取得タスクを作成します。イベントグループを通じて、取得プロセスの開始、停止、表示切り替えを安全に制御し、システムの安定動作を確保します。
- LVGLキャンバスに直接接続することで、カメラ画像のリアルタイム表示を実現します。ゼロコピー更新方式を採用し、インターフェースの表示/非表示切り替えをサポートしています。
- 外部にシンプルで使いやすいAPIを提供し、次のようなすべての機能を可能にします。
カメラの初期化、起動、および表示制御は、簡単な呼び出しだけで完了します。
- 必要なのは、私たちが作成したインターフェースをいつ呼び出すかを知ることだけです。
- 次に、「bsp_camera」コンポーネントの理解に焦点を当てましょう。
- まず、下記のGitHubリンクをクリックして、このレッスンのコードをダウンロードしてください。
https://github.com/Elecrow-RD/CrowPanel-Advanced-7inch-ESP32-P4-HMI-AI-Display-1024x600-IPS-Touch-Screen/tree/master/example/V1.1/idf-code/Lesson13-Camera_Real-Time
- 次に、このレッスンのコードをVS Codeにドラッグして、プロジェクトファイルを開きます。
- 開くと、このプロジェクトの枠組みが表示されます。

```

> .vscode
> build
  main
    include
  CMakeLists.txt
  idf_component.yml
  main.c
  managed_components
  peripheral
    bsp_camera
      include
      bsp_camera.h
      bsp_camera.c
  CMakeLists.txt
  sc2336_custom.json
  bsp_illuminate
.clangd
.gitignore
CMakeLists.txt
dependencies.lock
partitions.csv
README.md
sdkconfig
sdkconfig.old

```

このレッスンの例では、「peripheral」の下に「bsp_camera」という名前の新しいフォルダが作成されています。

「bsp_camera」フォルダ内に、新しい「include」フォルダと「CMakeLists.txt」ファイルが作成されました。

「bsp_camera」フォルダにはドライバファイル「bsp_camera.c」が、「include」フォルダにはヘッダーファイル「bsp_camera.h」が含まれています。

中でも、「sc2336_custom.json」ファイルは、簡単に言えば「SC2336 カメラ専用の設定ファイル」です。このファイルの目的は、一般的なデフォルト設定ではなく、プロジェクトで使用するハードウェアと要件に基づいてカメラを動作させることです。

「CMakeLists.txt」ファイルはドライバをビルドシステムに統合し、プロジェクトが「bsp_camera.c」に記述されたカメラの初期化および関連する表示機能を利用できるようにします。

カメラ表示コード

- カメラ表示コードは、「bsp_camera.c」と「bsp_camera.h」の2つのファイルで構成されています。
- 次に、まず「bsp_camera.h」プログラムを分析します。
- 「bsp_camera.h」はカメラ表示用のヘッダーファイルで、主に以下の目的で使用されます。
 - 「bsp_camera.c」で実装されている関数、マクロ、変数を宣言して使用する。
外部プログラムによる。
- 他の .c ファイルからこのモジュールを呼び出すには、`#include "bsp_camera.h"` を追加するだけです。
- 言い換えれば、どの機能と
定数は外部から利用可能だが、モジュールの内部詳細は隠蔽されている。
- このコンポーネントでは、使用に必要なすべてのライブラリが「bsp_camera.h」に含まれています。
統合管理のためのファイル。

```

4  #include <string.h>
5  #include "esp_log.h"
6  #include "esp_err.h"
7  #include "freertos/FreeRTOS.h"
8  #include "freertos/task.h"
9  #include "driver/i2c_master.h"
10 #include "esp_sccb_intf.h"
11 #include "esp_sccb_i2c.h"
12 #include "esp_cam_sensor.h"
13 #include "esp_cam_sensor_detect.h"
14 #include "linux/videodev2.h"
15 #include "esp_video_device.h"
16 #include "esp_video_init.h"
17 #include "esp_video_ioctl.h"
18 #include <sys/errno.h>
19 #include <sys/ioctl.h>
20 #include <sys/mman.h>
21 #include <sys/param.h>
22 #include <fcntl.h>
23 #include "esp_cache.h"
24 #include "esp_heap_caps.h"
25 #include "esp_private/esp_cache_private.h"
26 #include "bsp_illuminate.h"

```

- 例えば「esp_sccb_intf.h」、「esp_sccb_i2c.h」、「esp_cam_sensor.h」など
「esp_cam_sensor_detect.h」など（これらはすべてネットワークコンポーネントの下にあるライブラリです）。

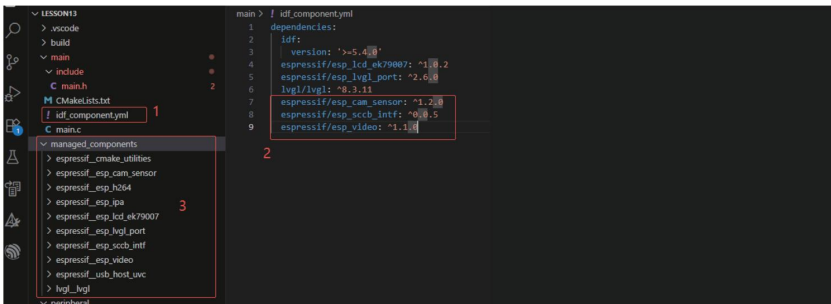
```

3  /*----- Header file -----*/
4  #include <string.h>
5  #include "esp_log.h"
6  #include "esp_err.h"
7  #include "freertos/FreeRTOS.h"
8  #include "freertos/task.h"
9  #include "driver/i2c_master.h"
10 #include "esp_sccb_intf.h"
11 #include "esp_sccb_i2c.h"
12 #include "esp_cam_sensor.h"
13 #include "esp_cam_sensor_detect.h"
14 #include "linux/videodev2.h"
15 #include "esp_video_device.h"
16 #include "esp_video_init.h"

```

- メインフォルダ内の「idf_component.yml」ファイルに、「esp_cam_sensor」、「esp_sccb_intf」、「esp_video」のバージョンを入力する必要があります。

- これらは公式ライブラリなので、カメラを実装するにはこれらに頼る必要があります
Advance-P4の機能について。



- 後続のコンパイル時に、プロジェクトは自動的にダウンロードします
esp_cam_sensorライブラリのバージョン1.2.0、esp_sccb_intfのバージョン0.0.5、およびesp_videoのバージョン1.1.0が必要です。ダウンロードが完了すると、これらのネットワークコンポーネントは「managed_components」フォルダに保存されます（このフォルダはバージョン番号を入力すると自動的に生成されます）。
- 次に、使用する変数と、具体的な実装が「bsp_camera.c」にある関数を宣言する必要があります。
- これらの宣言を「bsp_camera.h」に集約することで、呼び出しと管理が容易になります。（「bsp_camera.c」で使用される際の具体的な役割については後述します。）

```

peripheral > bsp_camera > include > C: bsp_camera.h > ...
11 #ifndef BSP_CAMERA_H
12 #define CAMERA_INFO(fmt, ...) ESP_LOGI(CAMERA_TAG, fmt, ##_VA_ARGS_)
13 #define CAMERA_DEBUG(fmt, ...) ESP_LOGD(CAMERA_TAG, fmt, ##_VA_ARGS_)
14 #define CAMERA_ERROR(fmt, ...) ESP_LOGE(CAMERA_TAG, fmt, ##_VA_ARGS_)
15
16 #define SCCB_MASTER_PORT 1
17 #define SCCB_GPIO_SCL 13
18 #define SCCB_GPIO_SDA 12
19
20 #define MAX_BUFFER_COUNT 10
21
22 typedef void (*camera_video_frame_operation_cb_t)(uint8_t *camera_buf, uint8_t camera_buf_index, uint32_t camera_buf_hes, uint32_t camera_buf_ves, size_t camera_buf_len);
23
24 typedef enum
25 {
26     VIDEO_TASK_DELETE = BIT(0),
27     VIDEO_TASK_DELETE_DONE = BIT(1),
28     VIDEO_TASK_DISPLAY_EN = BIT(2),
29 } video_event_id_t;
30
31 typedef struct
32 {
33     uint8_t *camera_buffer[MAX_BUFFER_COUNT];
34     size_t camera_buf_size;
35     uint32_t camera_buf_hes;
36     uint32_t camera_buf_ves;
37     struct v4l2_buffer v4l2_buf;
38     uint8_t camera_mcu_mode;
39     camera_video_frame_operation_cb_t user_camera_video_frame_operation_cb;
40     TaskHandle_t video_stream_task_handle;
41     EventGroupHandle_t video_event_group;
42 } camera_video_t;
43
44 esp_err_t camera_video_init();
45 esp_err_t camera_video_set_bufs(int video_fd, uint32_t fb_num, const void **fb);
46 esp_err_t camera_video_get_bufs(int fb_num, void **fb);
47 uint32_t app_video_get_buf_size(void);
48
49 esp_err_t video_stream_task_start(int video_fd, int core_id);
50 esp_err_t video_stream_task_stop(int video_fd);
51
52 esp_err_t video_register_frame_operation_cb(camera_video_frame_operation_cb_t operation_cb);
53 esp_err_t video_stream_wait_stop(void);
54
55 int camera_work();
56
57 void set_camera_img_display(bool state);

```

- 「bsp_camera.c」内の各関数の具体的な機能を見ていきましょう。
- 「bsp_camera」コンポーネントは、誰もが後々カメラを利用できるよう、重要なサポートを提供します。各機能の役割を明確に理解することで、カメラを便利に利用できるようになります。
- ここではコードの詳細な説明は行いません。各関数が何をするのか、そしてどのような状況で呼び出すべきなのかだけをお伝えします。

1. camera_video_init()

機能説明：

- この関数は、カメラの低レベルハードウェア初期化を完了します。SCCB(I2C)通信バスを作成および構成し、ESP32 CSIカメラインターフェースを初期化することで、メインコントローラとカメラセンサー間の基本的な通信リンクを確立します。

呼び出しタイミング：

- これは、ビデオデバイスを開いたり、フレームバッファを割り当てたり、ストリーミングタスクを開始したりする前の、システム初期化の最も早い段階で呼び出す必要があります。これは、カメラモジュールの基本的な初期化エントリです。

2. video_open()

機能説明：

- 標準のV4L2フレームワークに基づいてカメラデバイスノードを開き、ハードウェアの機能と解像度情報を照会し、出力ピクセルフォーマットをLVGLレンダリングとネイティブに互換性のあるRGB565に強制的に構成し、水平および垂直の画像ミラーリング構成をサポートします。

呼び出しタイミング：

- camera_video_init() の終了後、camera_work() 内で自動的に呼び出されます。低レベルの初期化処理です。上位アプリケーション層からこの関数を直接呼び出すことは推奨されません。

3. camera_video_set_bufs()

機能説明：

- カメラドライバから指定された数のビデオフレームバッファを適用し、USERPTRなどのユーザー定義メモリモードをサポートし、バッファクエリ、メモリバインディング、エンキュー操作を完了し、連続的なカメラフレームキャプチャのための循環バッファキューを構築します。

呼び出しタイミング：

- カメラデバイスが正常に開かれた後、ビデオストリームを開始する前に呼び出してください。これは、画像キャプチャ用のフレームバッファを準備するために必要な設定手順です。

4. camera_video_get_bufs()

機能説明：

- 初期化された内部フレームバッファのポインタリストを取得し、それを出力します。
外部モジュールにより、上位層のデバッグ、カスタム画像処理、および生データへのアクセスが容易になります。

呼び出しタイミング：

- バッファ構成が完了した後であればいつでも呼び出すことができます。デバッグ、カスタム画像アルゴリズム処理、生データのコピーなどに一般的に使用され、通常の表示ワークフローでは必要ありません。

5. app_video_get_buf_size()

機能説明：

- 現在のカメラに応じて、1フレームの合計バイトサイズを計算します。
解像度設定により、メモリ割り当て、データ伝送、フレームバッファ判定のための正確な長さ基準値を提供する。

呼び出しタイミング：

- カメラの解像度が確認された後に呼び出してください。メモリへの適用やデータ処理のために、最新の単一フレームバッファサイズを取得するためにいつでも呼び出すことができます。

6. video_receive_video_frame()

機能説明：

- カメラドライバのバッファキューからキャプチャされたフレームを1つ取り出し、フレームのインデックスと長さ情報を取得します。これは、新しいカメラフレームを取得するためのコア操作です。

呼び出しタイミング：

- ビデオストリーミングタスクのループ内で、新しいフレームを取得する必要があるたびに自動的に呼び出されます。これは、フレームキャプチャワークフロー全体の最初のステップです。

7. video_operation_video_frame()

機能説明：

- 登録済みのユーザーフレームコールバック関数を呼び出し、画像バッファアドレス、バッファインデックス、解像度、フレーム長を上位レイヤーに渡して、後続の表示またはアルゴリズム処理に使用します。

呼び出しタイミング：

- 新しいフレームを正常に受信した直後に呼び出され、生データを配信するために使用されます。
画像データをLVGLディスプレイまたはカスタムビジネスモジュールに変換します。

8. video_free_video_frame()

機能説明：

- 処理されたフレームバッファをドライバキューに再エンキューすることで、次のフレームのキャプチャのためにバッファを再利用し、継続的かつ安定したビデオストリーミングを維持するためのハードウェア。

呼び出しタイミング：

- 循環的な再利用を確実にするため、1フレームのデータ処理が完了した直後に呼び出す必要があります。フレームバッファと途切れることのないカメラキャプチャを実現します。

9. video_stream_start()

機能説明：

- カメラドライバに制御コマンドを送信してハードウェアビデオストリーミングを有効にします。カメラが画像データを連続的に出力し始めるようにする。

呼び出しタイミング：

- バッファ構成が完了した後、ストリーミングタスクを作成する前に呼び出します。ハードウェアによる画像キャプチャ処理を正式に開始します。

10. video_stream_stop()

機能説明：

- カメラハードウェアのビデオストリームを停止し、キャプチャパイプラインを閉じ、タスク停止完了をマークし、安全なリソースシャットダウンのためにフラグを立てる。

呼び出しタイミング：

- カメラの電源を切る、ストリーミング タスクを終了する、またはシャットダウンする必要がある場合に呼び出します。システムは通常はタスク停止プロセスによって内部的に実行される。

11. video_stream_task()

機能説明：

- これは、FreeRTOSのビデオストリーミングにおけるコアとなるバックグラウンドタスクです。フレームの受信、フレームコールバック処理、バッファの再キューイングを実行するために無限ループを実行し、イベントグループを使用してタスクの開始、停止、および表示スイッチの状態を制御します。

呼び出しタイミング：

- video_stream_task_start() によってバックグラウンドとして自動的に作成およびスケジュールされます。デモントask。上位層アプリケーションからの手動呼び出しは不要です。

12. video_stream_task_start()

機能説明：

- イベントグループを初期化し、指定されたCPUコアに固定されたビデオストリーミングタスクを作成し、ハードウェアビデオストリームが連続カメラキャプチャモードに入るようにします。

呼び出しタイミング：

- デバイスの起動、バッファの設定、コールバックの登録がすべて完了し、カメラのストリーミングワークフロー全体が正式に開始された後に呼び出します。

13. video_stream_task_stop()

機能説明：

- イベントグループフラグを設定して、ビデオストリーミングタスクに停止信号を送信し、タスクを安全に終了し、カメラキャプチャワークフローを正常に閉じます。

呼び出しタイミング：

- システムシャットダウン、再起動、機能切り替え、またはカメラの電源オフ時に呼び出します
必須。

14. video_register_frame_operation_cb()

機能説明：

- カスタムフレーム処理コールバック関数を登録し、基盤となるカメラキャプチャロジックを、表示、アルゴリズム分析、データ送信などの上位層のビジネスロジックから分離します。

呼び出しタイミング：

- カメラの初期化が完了した後、ビデオストリーミングタスクを開始する前に呼び出して、カスタムフレーム処理関数を事前にバインドします。

15. video_stream_wait_stop()

機能説明：

- ビデオストリーミングタスクが完全に終了するまでブロックして待機し、すべてのタスクが確実に終了するようにします。
リソースは完全に解放され、強制終了によるシステムクラッシュやメモリリークが回避されます。

呼び出しタイミング：

- `video_stream_task_stop()` を呼び出した後、システムシャットダウン前に呼び出してください。
タスクが安全に終了したことを確認するために、再起動してください。

16. camera_video_frame_operation()

機能説明：

- これはデフォルトの組み込みフレームコールバック関数です。カメラフレームバッファをLVGLキャンバスに直接バインドしてゼロコピーのリアルタイム表示を実現し、LVGLスレッドロック保護を追加してマルチスレッドGUIのリフレッシュの安全性を確保します。

呼び出しタイミング：

- ストリーミングタスクによって新しいフレームが1つキャプチャされるたびに自動的にトリガーされます。ユーザーによる手動呼び出しは不要です。

17. camera_work()

機能説明：

- これは、カメラを有効にするためのワンクリック入力機能です。デバイスの起動、SPIRAMメモリの割り当て、コールバックの登録、LVGLキャンバスの作成、バッファの設定、ストリーミングタスクの起動など、カメラの起動手順をすべて内部的に実行します。

呼び出しタイミング：

- システム初期化時に camera_video_init() の直後に呼び出してください。これはカメラアプリケーション全体を起動するための主要な上位層APIです。

18. set_camera_img_display()

機能説明：

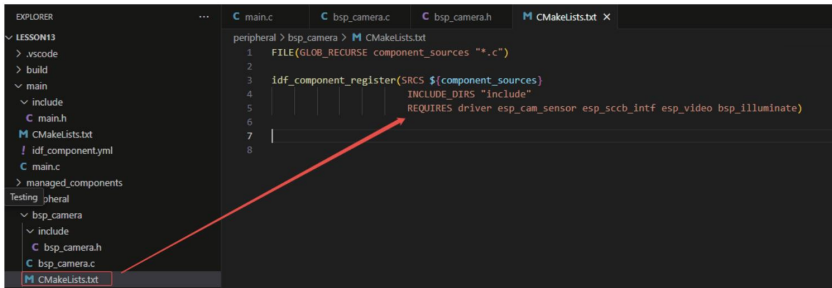
- カメラのLVGL表示ウィジェットの表示/非表示を制御し、同時にフレーム処理の有効化/無効化を行います。基となるキャプチャタスクを停止することなく、画面プレビューのオン/オフを切り替えることができます。

呼び出しタイミング：

- システム実行中いつでも呼び出し可能で、シーン切り替え、ディスプレイ非表示、省電力機能と機能のオン/オフ制御。

これでbsp_cameraコンポーネントの紹介は終了です。皆さんの目的には、これらのインターフェースの呼び出し方法を知っていれば十分です。

これらのインターフェースを呼び出す必要がある場合は、bsp_cameraフォルダの下にある「CMakeLists.txt」ファイルも構成する必要があります。bsp_cameraフォルダにあるこのファイルは、主にESP-IDFビルドシステム（CMake）にbsp_cameraコンポーネントをコンパイルおよび登録する方法を指示する役割を果たします。



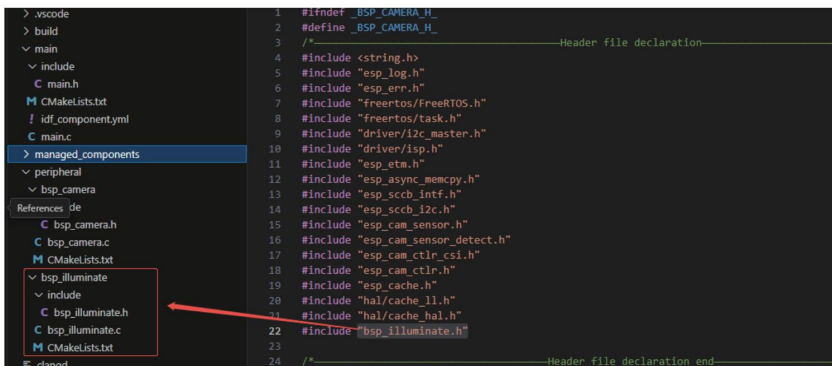
- 「driver」、「esp_cam_sensor」、「esp_sccb_intf」、「esp_video」、「bsp_illuminate」を含める理由は、「bsp_camera.h」でこれら呼び出しているためです（システムライブラリであるその他のライブラリは追加する必要はありません）。

```

3  /*----- Header file declaration-----
4  #include <string.h>
5  #include "esp_log.h"
6  #include "esp_err.h"
7  #include "freertos/FreeRTOS.h"
8  #include "freertos/task.h"
9  #include "driver/i2c_master.h"
10 #include "esp_sccb_intf.h"
11 #include "esp_sccb_i2c.h"
12 #include "esp_cam_sensor.h"
13 #include "esp_cam_sensor_detect.h"
14 #include "linux/video_dev2.h"
15 #include "esp_video_device.h"
16 #include "esp_video_init.h"
17 #include "esp_video_ioctl.h"
18 #include <sys/errno.h>
19 #include <sys/ioctl.h>
20 #include <sys/mman.h>
21 #include <sys/param.h>
22 #include <fcntl.h>
23 #include "esp_cache.h"
24 #include "esp_heap_caps.h"
25 #include "esp_private/esp_cache_private.h"
26 #include "bsp_illuminate.h"

```

- 例えば、「bsp_illuminate.h」は画面表示に関連するコンポーネントで、既に説明済みです。以前に詳しく説明しましたので、ここでは改めて触れません。
- 画面の初期化、画面のバックライトの点灯、および画面に関連コンテンツを表示させるために使用されます。



主な機能

- メインフォルダはプログラム実行のコアディレクトリであり、メインファイルが含まれています。
関数実行ファイル main.c

- ビルドシステムの「CMakeLists.txt」ファイルにメインフォルダを追加します。

```

31 void init(void) // System initialization function
43     esp_ldo_channel_config_t ldo4_cof = { // LDO channel 4 configuration
47         err = esp_ldo_acquire_channel(&ldo4_cof, &ldo4); // Acquire and configure LDO4 channel
48         if (err != ESP_OK) // Check for error
49             init_fail("ldo4", err); // Handle initialization failure
50
51         err = gpio_install_isr_service(0); // Install GPIO interrupt service routine
52         if (err != ESP_OK) // Check for error
53             init_fail("gpio isr service", err); // Handle initialization failure
54
55         err = display_init(); // Initialize LCD display
56         if (err != ESP_OK) // Check for error
57             init_fail("display", err); // Handle initialization failure
58
59         err = set_led_blight(100); // Enable backlight with 100% brightness
60         if (err != ESP_OK) { // Check error
61             init_fail("LCD Backlight", err); // Handle failure
62         }
63         MAIN_INFO("LCD backlight opened (brightness: 100%); // Log success message for backlight
  
```

- これはアプリケーション全体のエントリファイルです。ESP-IDFにはint main()はありません。代わりに、プログラムはvoid app_main(void)から実行を開始します。

- まず、「main.c」について説明しましょう。

- プログラムの実行時の一般的な処理は以下のとおりです。

- システムが app_main エントリで起動した後、Init 関数を呼び出して完全な初期化を完了します。まず、カメラと LCD の LDO 電源を設定し、GPIO 割り込みサービスを初期化し、LCD ディスプレイを初期化してバックライトをオンにし、次に camera_video_init を呼び出してカメラ ハードウェアを初期化し、camera_work を使用してカメラ デバイスを開き、フレーム バッファを割り当て、ディスプレイ コールバックを登録し、ビデオ キャプチャ タスクを開始します。

- 初期化後、set_camera_img_display(true)が実行され、カメラプレビューが有効になります。これにより、ストリーミング タスクはコールバック関数を介してキャプチャした画像データをリアルタイムでLVGLキャンバスに送信できるようになり、画面上に連続的かつリアルタイムなカメラ表示が実現します。

- 次に、メインコード「main.c」について説明しましょう。

```

1  /*-----Header file declaration-----*/
2  #include "main.h" // Include the main header file containing required definitions and declarations
3  /*-----Header file declaration end-----*/
  
```

- これには、通常ログマクロ、周辺機器の初期化宣言、および必要な他のインターフェースのヘッダーファイルを含むカスタムメインヘッダーファイル「main.h」が含まれます。
使用される。

- 以下は「main.h」の内容です。

```

~ LESSON13
> .vscode
> build
> main
  > include
    C:main.h
  M:CMakeLists.txt
  ! idf_component.yml
  C:main.c
> managed_components
  > peripheral
    > bsp_camera
      > include
        C:bsp_camera.h
      C:bsp_camera.c
  M:CMakeLists.txt
  > bsp_illuminate
    > include
      C:bsp_illuminate.h
      C:bsp_illuminate.c
  M:CMakeLists.txt
  > .clangd
  > .gitignore
  M:CMakeLists.txt
  > dependencies.lock

main > include > C:main.h ...
1  #ifndef MAIN_H
2  #define MAIN_H
3
4  /*-----Header file declaration-----*/
5  #include <stdio.h>
6  #include "string.h"
7  #include "freertos/FreeRTOS.h"
8  #include "freertos/task.h"
9  #include "esp_log.h"
10 #include "esp_err.h"
11 #include "esp_private/esp_clk.h"
12 #include "esp_lio_regulator.h"
13 #include "esp_sleep.h"
14 #include "driver/rtc_io.h"
15 #include "esp_timer.h"
16
17 #include "bsp_camera.h"
18 #include "bsp_illuminate.h"
19
20 /*-----Header file declaration end-----*/
21 /*-----Variable declaration-----*/
22 #define MAIN_TAG "MAIN"
23 #define MAIN_INFO(fmt, ...) ESP_LOGI(MAIN_TAG, fmt, ##__VA_ARGS__)
24 #define MAIN_DEBUG(fmt, ...) ESP_LOGD(MAIN_TAG, fmt, ##__VA_ARGS__)
25 #define MAIN_ERROR(fmt, ...) ESP_LOGE(MAIN_TAG, fmt, ##__VA_ARGS__)
26
27 /*-----Variable declaration end-----*/
28 #endif

```

- 「main.c」の内容について引き続き見ていきましょう。

- LDO電源制御ハンドル :周辺機器（カメラなど）に電源を供給するために使用されます。（LCDも含む）。

- ldo3は2.5V出力に対応します。

- ldo4は3.3V出力に対応します。

関数宣言:

- init_fail: 初期化の失敗を処理します。
- Init: システムハードウェアの初期化を実行します。

```

8  static esp_ldo_channel_handle_t ldo4 = NULL; // H
9  static esp_ldo_channel_handle_t ldo3 = NULL; // H
10
11 // function declaration
12 void init_fail(const char *name, esp_err_t err);
13 void Init(void); // Function declaration for syst

```

初期化失敗:

- 初期化失敗時の処理機能：
 - 静的ブール状態を使用して、繰り返し印刷を防ぎます。
 - 初期化失敗メッセージを表示しながら、無限ループで実行されます。
 - 1サイクルあたり1秒間の遅延。
- 機能 :ハードウェアの初期化に失敗すると、プログラムはそれ以上の実行を停止し、エラーメッセージを表示します。

```

34 void init_fail(const char *name, esp_err_t err) // Function to handle initialization failures
35 {
36     static bool state = false; // Flag to avoid repeated error logging
37     while (1) // Stay in infinite loop after failure
38     {
39         if (!state) // Print error message only once
40         {
41             MAIN_ERROR("%s init [ %s ]", name, esp_err_to_name(err)); // Log initialization failure with error name
42             state = true; // Update state to prevent repeated logs
43         }
44         vTaskDelay(1000 / portTICK_PERIOD_MS); // Wait 1 second before looping again
45     }
46 }

```

熱：

- Init(void) は、システム起動時にすべてのハードウェア モジュールを設定および起動してカメラ表示の準備を完全に整える。統一されたシステム初期化関数です。この関数は、LCD とカメラに安定した電力を供給するために LDO 電源チャネルを順次設定し、周辺機器の割り込みをサポートするために GPIO 割り込みサービスをインストールし、LCD ディスプレイを初期化してバックライトを 100% の明るさに設定して画面が正しく点灯するようにし、カメラの低レベル ハードウェア インターフェイスを初期化し、最後にカメラの動作フロー全体を起動します。初期化手順のいずれかが失敗した場合、この関数は無限のエラー処理ループに入り、失敗をログに記録してシステムが異常に動作しないようにします。

```

31 void Init(void) // System initialization function
32 {
33     static esp_err_t err = ESP_OK; // Variable to store function return values
34
35     esp_ldo_channel_config_t ldo3_cof = { // LDO channel 3 configuration
36         .chan_id = 3, // Channel ID: 3
37         .voltage_mv = 2500, // Output voltage: 2.5V
38     };
39     err = esp_ldo_acquire_channel(&ldo3_cof, &ldo3); // Acquire and configure LDO3 channel
40     if (err != ESP_OK) // Check for error
41         init_fail("ldo3", err); // Handle initialization failure
42
43     esp_ldo_channel_config_t ldo4_cof = { // LDO channel 4 configuration
44         .chan_id = 4, // Channel ID: 4
45         .voltage_mv = 3300, // Output voltage: 3.3V
46     };
47     err = esp_ldo_acquire_channel(&ldo4_cof, &ldo4); // Acquire and configure LDO4 channel
48     if (err != ESP_OK) // Check for error
49         init_fail("ldo4", err); // Handle initialization failure
50
51     err = gpio_install_isr_service(0); // Install GPIO interrupt service routine
52     if (err != ESP_OK) // Check for error
53         init_fail("gpio isr service", err); // Handle initialization failure
54
55     err = display_init(); // Initialize LCD display
56     if (err != ESP_OK) // Check for error
57         init_fail("display", err); // Handle initialization failure
58
59     err = set_lcd_blight(100); // Enable backlight with 100% brightness
60     if (err != ESP_OK) { // Check error
61         init_fail("LCD Backlight", err); // Handle failure
62     }
63     MAIN_INFO("LCD backlight opened (brightness: 100)"); // Log success message for backlight
64
65     err = camera_video_init();
66     if (err != ESP_OK)
67         init_fail("camera", err);
68     video_node = camera_work();
69     if (-1 == video_node)
70         init_fail("camera", ESP_FAIL);
71 }

```

app_main:

- `app_main(void)` は、ESP32 アプリケーション全体のメインエントリーポイントであり、プログラムの開始点として機能します。最初に起動ログを出力し、次に `Init()` 関数を呼び出して、電源、LCD、GPIO、カメラを含むすべてのハードウェアの統合初期化を完了します。すべてのハードウェアの準備が完了すると、`set_camera_img_display(true)` を呼び出してカメラのプレビュー表示を有効にし、最後にプロンプトログを出力して、画面にリアルタイムのカメラ画像が正常に表示されていることを示します。

```

73 void app_main(void) // Main application entry point
74 {
75     MAIN_INFO("-----Camera task-----\r\n"); // Print sta
76
77     Init(); // Call system initialization function
78
79     set_camera_img_display(true); // Enable camera image display
80
81     MAIN_INFO("-----The screen is displaying.-----\r\n");
82 }

```

- 最後に、メインディレクトリにある「CMakeLists.txt」ファイルを見てみましょう。
- このCMake設定の役割は以下のとおりです。
 - `main/` ディレクトリ内のすべての `.c` ソースファイルをコンポーネントのソースファイルとして収集します。
 - メインコンポーネントをESP-IDFビルドシステムに登録し、カスタムコンポーネント「`bsp_camera`」とカスタムコンポーネント「`bsp_illuminate`」に依存することを宣言します。
- このように、ビルドプロセス中に、ESP-IDF は「`bsp_camera`」をビルドし、まず「`bsp_illuminate`」をビルドし、次に「`main`」をビルドします。



```

main > M CMakeLists.txt
1 FILE(GLOB_RECURSE main ${CMAKE_SOURCE_DIR}/main/*.c)
2
3 idf_component_register(SRCS ${main}
4                       INCLUDE_DIRS "include"
5                       REQUIRES bsp_illuminate bsp_camera)
6

```

注：以降のコースでは、新しい「CMakeLists.txt」ファイルをゼロから作成することはありません。代わりに、既存のファイルに若干の変更を加えて、他のドライバをメイン関数に統合します。

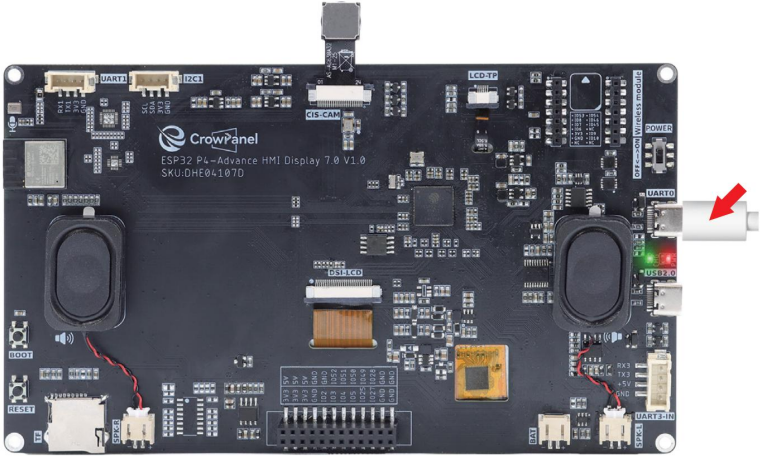
完全なコード

完全なコード実装をご覧になるには、下記のリンクをクリックしてください。

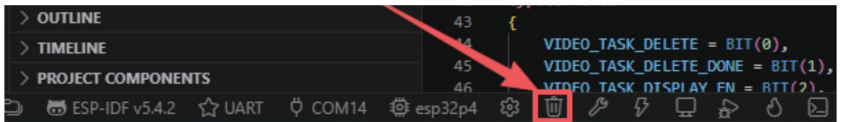
https://github.com/Elecrow-RD/CrowPanel-Advanced-7inch-ESP32-P4-HMI-AI-Display-1024x600-IPS-Touch-Screen/tree/master/example/V1.1/idf-code/Lesson13-Camera_Real-Time

プログラミング手順

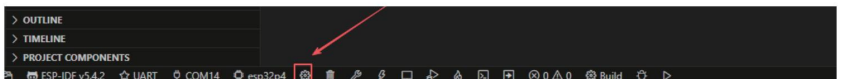
- コードの準備ができたので、次のステップはそれを ESP32-P4 に書き込むことです。
実際の行動を観察する。
- まず、USBケーブルを使ってAdvance-P4デバイスをコンピュータに接続してください。



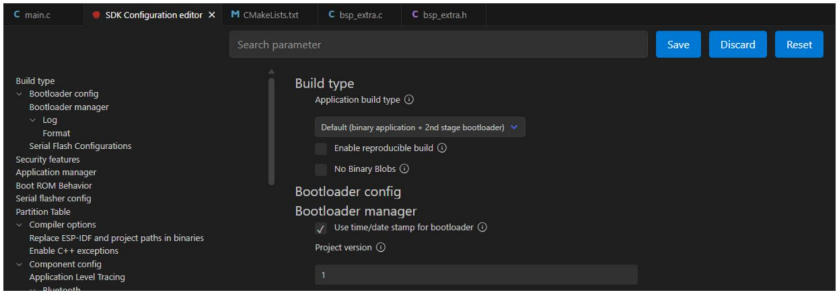
- フラッシュ準備を開始する前に、コンパイル済みのファイルをすべて削除して、プロジェクトを初期の「未ビルド」状態に戻してください。（これにより、以降のコンパイルが以前のビルド成果物の影響を受けないことが保証されます。）



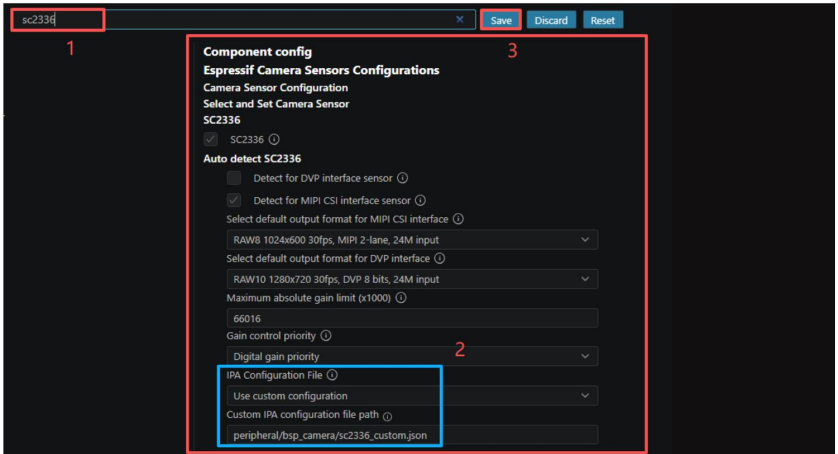
- まず、最初のセクションの手順に従って、ESP-IDFのバージョン、コードアップロード方法、シリアルポート番号、およびターゲットチップを正しく選択してください。
- 次に、SDKを設定する必要があります。
- 下の画像に表示されているアイコンをクリックしてください。



- しばらく読み込みが続くと、関連するSDKに進むことができます。
構成。



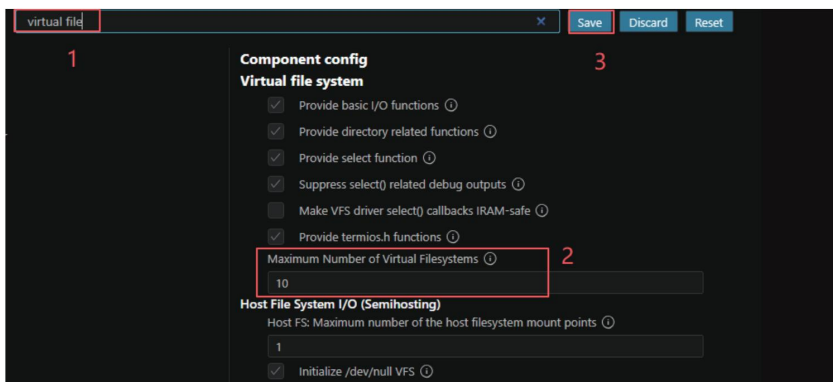
- このプロジェクトでは画面に情報を表示するため、まずレッスン7のSDK構成を参照して画面表示設定を完了し、画面が正しく機能することを確認することをお勧めします（詳細は101～103ページを参照してください）。
- ここでは、カメラを正常に使用するために、カメラの設定オプションを構成する必要があります。（写真に示されているのと同じ構成にしてください。）



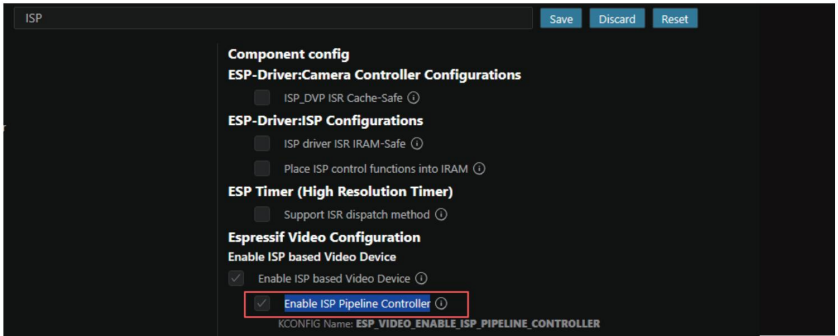
- 上記の2番目の手順では、この目的のために用意した設定ファイルを使用します。前述のとおり、カメラの画面表示を正常にするには、この方法を用いる必要があります。



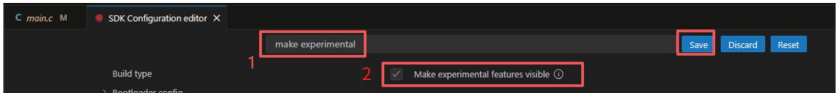
- 次に、「仮想ファイル」を検索し、この値を10に設定します。他の数値でも構いません。
- ESP-IDFのSDK構成では、「仮想ファイルシステムの最大数」を使用して、VFSフレームワークが同時にマウントできるファイルシステム/デバイスノードの最大数を設定します。
- プロジェクトでは「/dev/video0」のような V4L2 ビデオ デバイス ノードを使用するだけでなく、シリアルポート、ログ、SD カード、フラッシュ ファイルシステムなどの他の VFS デバイスをマウントすることもあるため、デフォルトの構成値では通常、これらすべてのマウント ポイントに対応するには不十分です。
- したがって、この値を10に設定すると、問題を回避するために十分なVFSスロットが確保されます。デバイス登録の失敗、カメラが正常に起動しない、マウントポイント不足によるシステム動作の不安定化など、様々な問題が発生します。この対策により、プロジェクト内でVFS経由でアクセスする必要のあるすべてのデバイスとファイルシステムが正常に登録され、使用できることが保証されます。



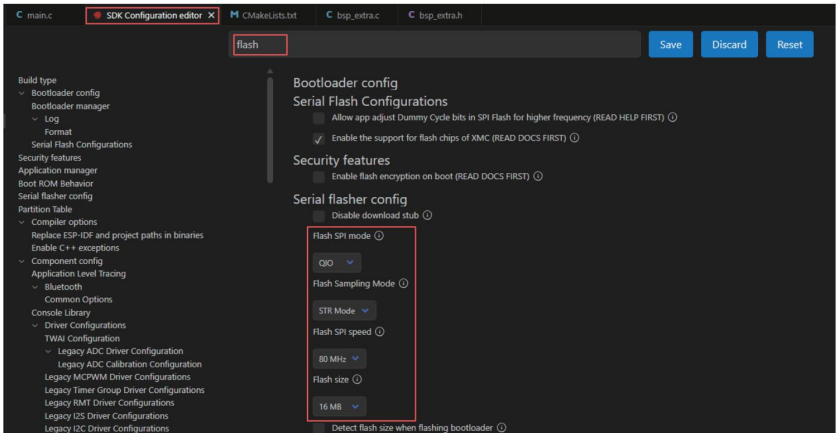
- 次に、「ISP」を検索して、画像内のこれを開きます。
- ESP-IDFでは、「ISPベースのビデオデバイスを有効にする」と「ISPパイプラインコントローラを有効にする」が、カメラISP（画像信号プロセッサ）機能の中核となる設定です。これらを実効にすると、システムはハードウェアISPに基づいてビデオデバイスドライバをアクティブ化し、独立したisp_task/バックグラウンドタスクを作成します。このタスクは、ISPハードウェアから露出、ホワイトバランス、その他の統計情報を自動的に読み取り、画像処理アルゴリズムを実行し、カメラまたはISPパラメータを動的に調整して、自動露出や自動ホワイトバランスなどの画質最適化を実現します。
- これにより、カメラからの出力画像の色と明るさがより安定し、自然な仕上がりになるだけでなく、アプリケーション層で複雑な画像調整ロジックを手動で記述する必要もなくなります。これらは、ESP32-P4上でSC2336やその他のMIPIカメラを使用して高品質な画像を取得するために不可欠な設定です。



- 次に、実験的機能オプションを有効にします。そうすることで、200MHz PSRAM。



- その後、検索ボックスに「flash」と入力して、Flash関連の設定を探してください。（フラッシュの設定が私のものと完全に一致していることを確認してください。）



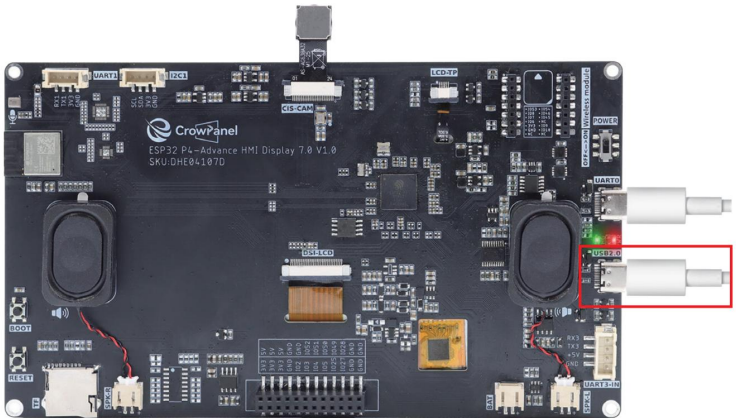
- 設定が完了したら、必ず設定を保存してください。

- 次に、コードをコンパイルしてフラッシュします（最初のレッスンで詳しく説明します）。

- ここでは、非常に便利な機能も紹介します。ボタンを1つ押すだけで、コンパイル、アップロード、モニターの有効化を一度に実行できます。



- コードのコンパイルとアップロードが完了するまでしばらくお待ちください。
モニターは自動的に開きます。
- この時点で、USB 2.0ポート経由でAdvance-P4をType-Cケーブルで接続することを忘れないでください。これは、コンピューターのUSB-Aポートから供給される最大電流が一般的に500mAであり、Advance-P4は複数の周辺機器（特に画面）を使用する際に十分な電力供給を必要とするためです。（充電器に接続することをお勧めします。）



- コードを実行すると、カメラからのリアルタイムフィードを次の場所で見ることができます。
Advance-P4の画面。

